

“up” image. When the user moves the cursor over a button, we show the “over” image and the code transitions to the over state. When the user clicks down on the button, we show the “down” image and the code moves into the down state (internally called the *armed* state). When the user releases the mouse button (clicks up), we again show the “over” image, and the code transitions back to the over state (and `handleEvent()` returns `True` to indicate that a click has happened). If the user then moves the cursor off the button, we show the “up” image again and transition back to the up state.

Next, I’ll show you how we can use a state machine to represent different scenes that a user can encounter in a larger program. As a generic example, we’ll have the following scenes: *Splash* (starting), *Play*, and *End*. We’ll create a set of constants that represent the different states, create a variable called `state`, and assign it the value of the starting state:

```
STATE_SPLASH = 'splash'
STATE_PLAY = 'play'
STATE_END = 'end'
state = STATE_SPLASH # initialize to starting state
```

In order to perform different actions in the different states, in the program’s main loop we use an `if/elif/elif/.../else` construct that branches based on the current value of the `state` variable:

```
while True:
    if state == STATE_SPLASH:
        # Do whatever you want to do in the Splash state here
    elif state == STATE_PLAY:
        # Do whatever you want to do in the Play state here
    elif state == STATE_END:
        # Do whatever you want to do in the End state here
    else:
        raise ValueError('Unknown value for state: ' + state)
```

Since `state` is initially set to `STATE_SPLASH`, only the first branch of the `if` statement will run.

The idea of a state machine is that under certain circumstances, typically triggered by some event, the program changes its state by assigning a different value to the `state` variable. For example, the starting Splash scene